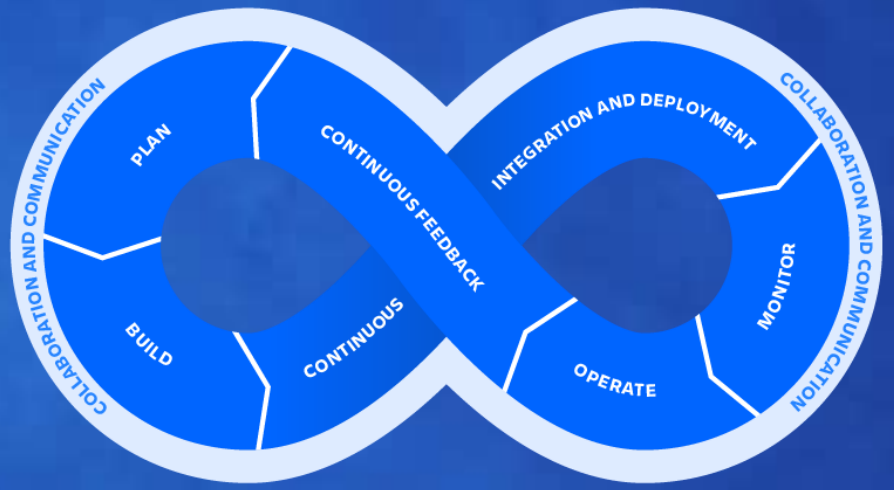


Session 5.
Working with Kubernetes Clusters

Ram N Sangwan

Agenda

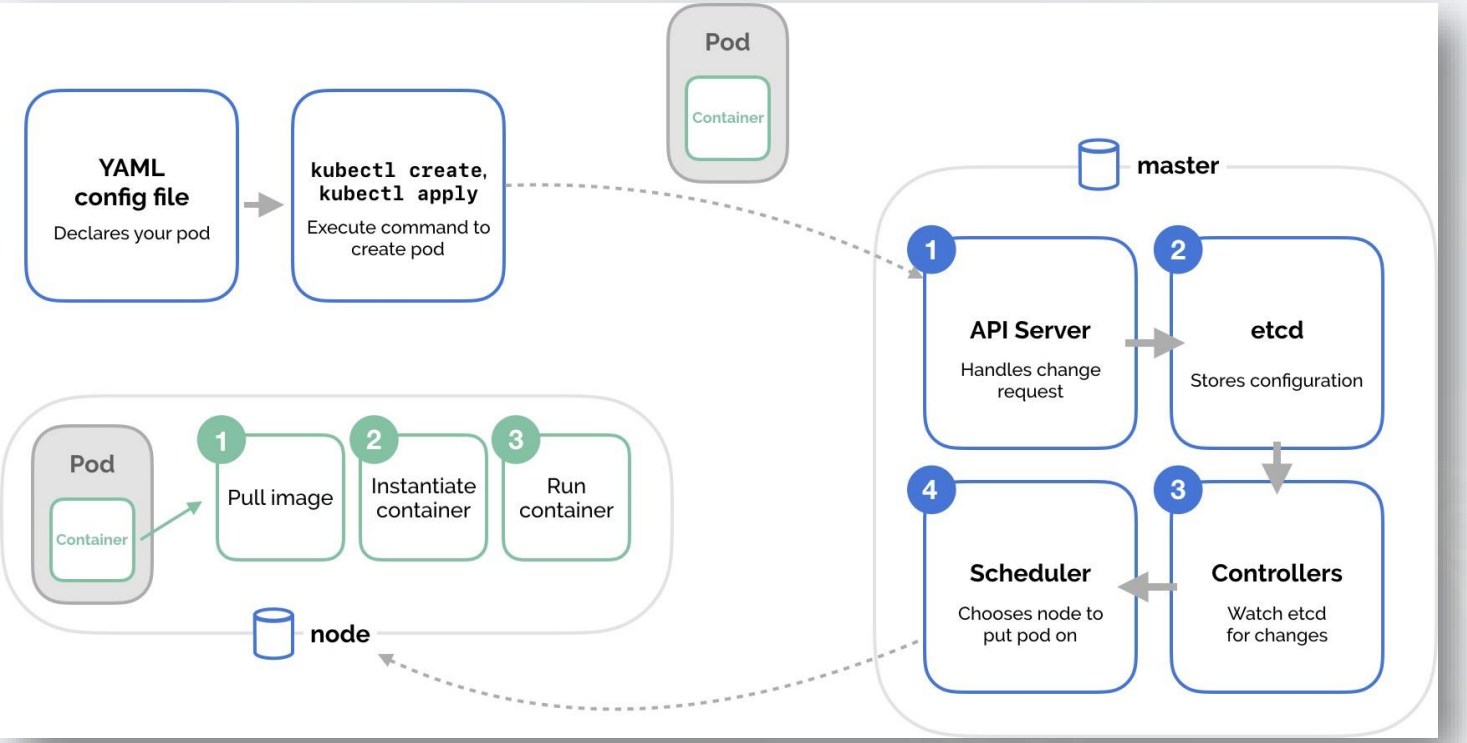
- Introduction to Pods and Pod Lifecycle
- Create & Manage Pods
- Deployments
- Service

Accessing a Cluster Using Kubectl

- Set up a Kubernetes configuration file, `config`, in ***$HOME/.kube***.
- Start using ***kubectl***.

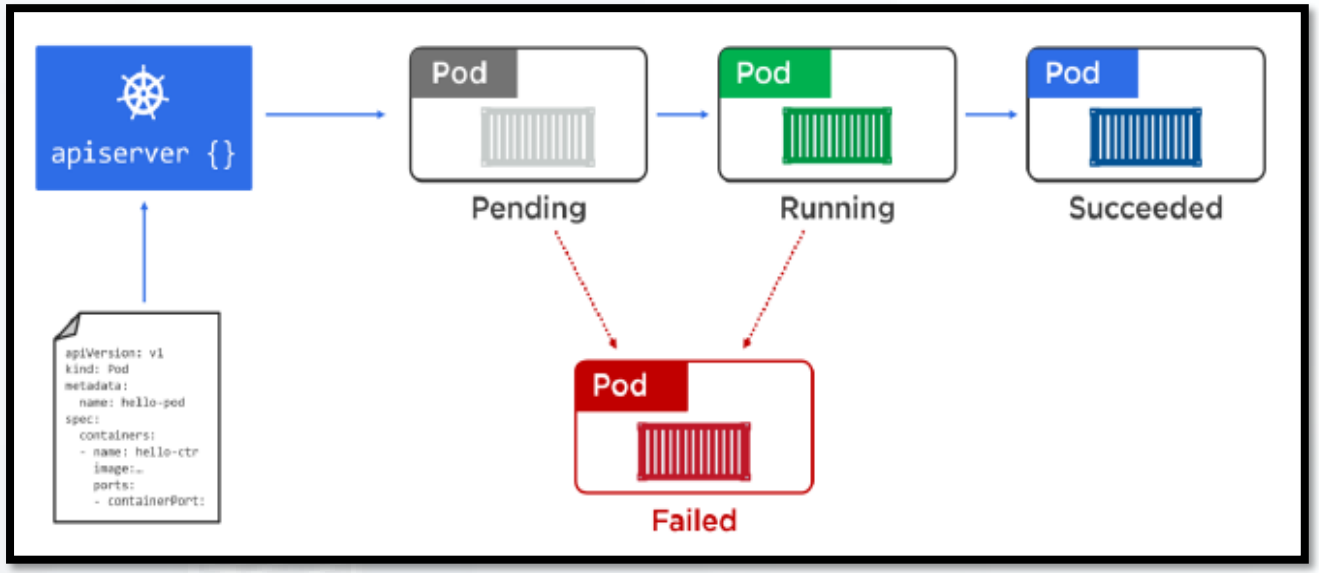
Pods

- While you can run multiple containers in a pod, in reality you often use one container per pod.
- When designing your pods, don't force multiple containers into the same pod
  - There are specific reasons to have multiple containers in a pod.



Pod lifecycle

- In its lifetime, a pod transitions through several stages.
- A pod's current stage is tracked in the ***status*** field, with a subfield ***phase***.
- When a pod's state changes, the **kubelet** process on that node updates the ***phase*** field in the pod's **etcd** entry.
- A pod has one of several phases:
  - Pending,
  - Running,
  - Succeeded,
  - Failed, or
  - Unknown.



Creating Pods – First Step

- Create Pod manifest file, pod.yaml:
- We can see four top-level resources:
  - apiVersion
  - kind
  - metadata
  - spec

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    zone: prod
    version: v1
spec:
  containers:
    - name: hello-ctr
      image: nigelpoulton/k8sbook:latest
      ports:
      - containerPort: 8080
```

- The .apiVersion field tells us *API group* and *API version* that will be used to create the object.
- Normally the format is <api-group>/<version>.
- Pods are defined in a special API group called the ***core*** group which omits the ***api-group*** part.

Creating Pods – Second Step

- **Deploying Pods from a manifest file**
  - Use the following kubectl command to POST the manifest to the API server.

    ```
    # kubectl apply -f pod.yml
    ```

- The pod will be deployed in the default namespace

```
[root@server kubernetes]# kubectl apply -f pod.yml
pod/hello-pod created
[root@server kubernetes]#
```

Information on Pods

- A pod's ***status*** column is shown when you run ***kubectl get pods***.
- This column provide a useful view of what has happened to the pod.

```
# kubectl get pod mypod -o yaml
# kubectl describe pod mypod
```

- You can add the --watch flag to the kubectl get pods command.

```
[root@server ~]# kubectl get pods --all-namespaces
NAMESPACE              NAME                                          READY   STATUS             RESTARTS      AGE
kube-system            coredns-78fcd69978-2tz88                      1/1     Running            3 (17m ago)   2d23h
kube-system            coredns-78fcd69978-6d6sw                      1/1     Running            3 (17m ago)   2d23h
kube-system            etcd-server.example.com                       1/1     Running            3 (17m ago)   2d23h
kube-system            kube-apiserver-server.example.com             1/1     Running            3 (17m ago)   2d23h
kube-system            kube-controller-manager-server.example.com    1/1     Running            4 (17m ago)   2d23h
kube-system            kube-flannel-ds-fcslq                         1/1     Running            3 (17m ago)   2d23h
kube-system            kube-flannel-ds-hk8nd                         1/1     Running            0             2d20h
kube-system            kube-flannel-ds-nm28w                         1/1     Running            3             2d20h
kube-system            kube-proxy-5zwjs                              1/1     Running            3 (17m ago)   2d23h
kube-system            kube-proxy-j5zx9                              1/1     Running            3 (17m ago)   2d20h
kube-system            kube-proxy-tg2sd                              1/1     Running            0             2d20h
kube-system            kube-scheduler-server.example.com             1/1     Running            3 (17m ago)   2d23h
kubernetes-dashboard   dashboard-metrics-scraper-856586f554-fzjj2    1/1     Running            3 (17m ago)   2d23h
kubernetes-dashboard   kubernetes-dashboard-79b875f7f8-rfh88         0/1     ImagePullBackOff   2 (17m ago)   2d23h
nsoracle               mypod                                         1/1     Running            1 (17m ago)   12h
[root@server ~]#
```

Introspecting and deleting Pod

- Log into it or execute commands in it.

```
# kubectl exec hello-pod -- ps aux
```

- Run the following command will log-in to the first container in the hello-container Pod.

```
# kubectl exec -it hello-pod -- sh
```

- Delete your pod by typing **exit** to quit your shell session inside the container, then run `kubectl delete` to delete the Pod.

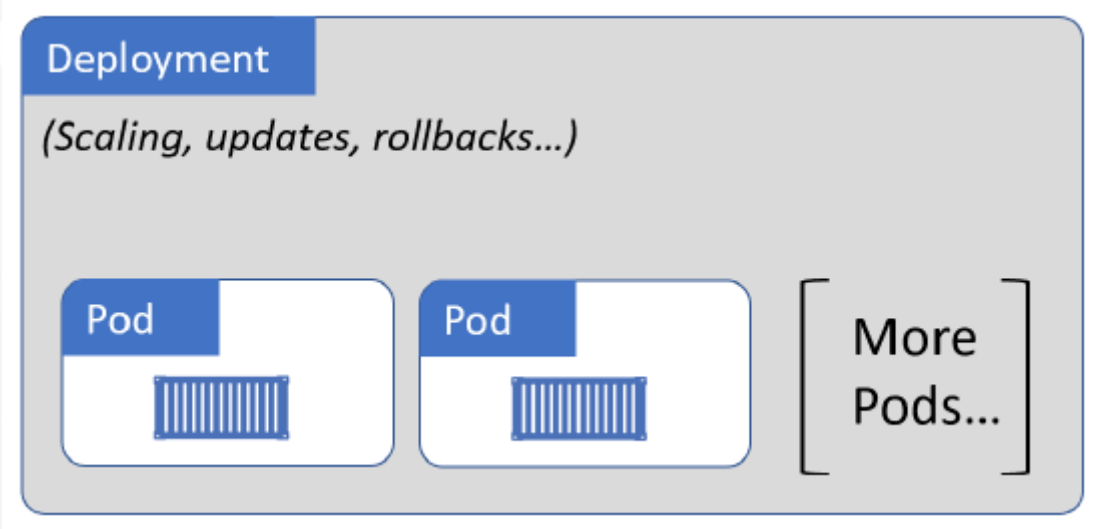
```
# exit

# kubectl delete -f pod.yml
```

- Watch [this]() comprehensive Lecture on Pods.

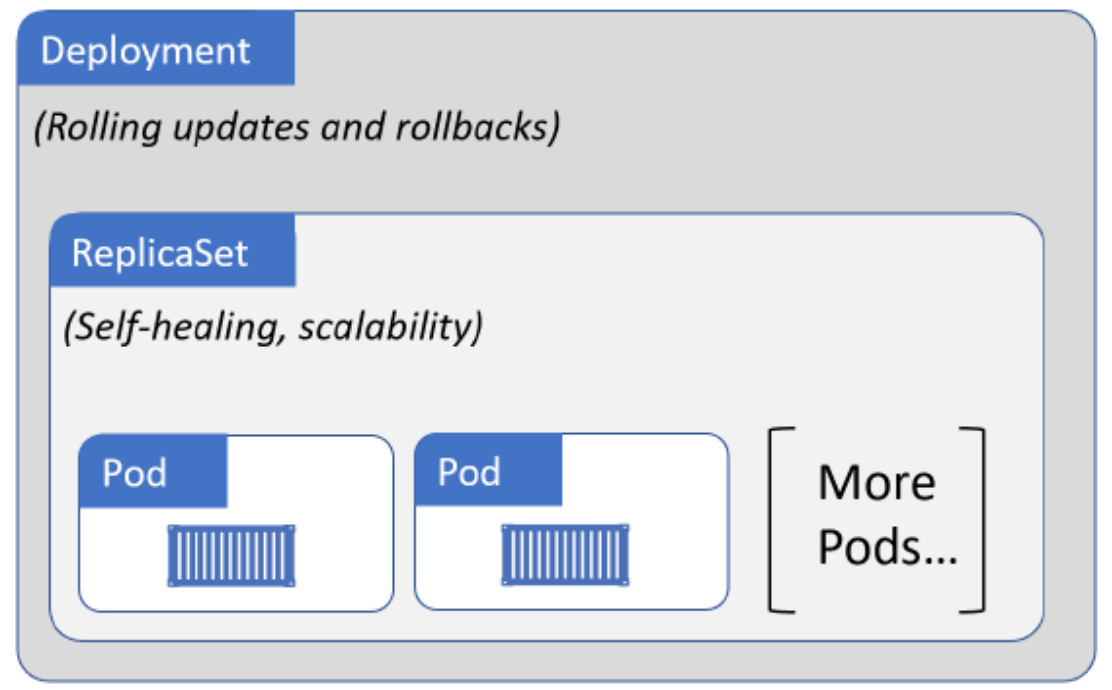
Deployments

- Deployments let you manage a set of identical pods through a single parent object - the deployment.
- If you have a pod that you want many copies of - say an application server that needs to scale up with increased load - deployments are the right tool.



Deployments

- Think of *Deployments* as managing *ReplicaSets*, and *ReplicaSets* as managing *Pods*.
- Put them all together, and we've got a great way to deploy and manage applications on Kubernetes



Deployment YAML

- In Kubernetes, you can declare your desired system state through configuration files which can be tracked, versioned, and easily understood.
- Like pods, deployments are declared through YAML files.
- Create a file `deploy.yml` in your system path with given code.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-pod
        image: nigelpoulton/k8sbook:latest
        ports:
        - containerPort: 8080
```

Deployment YAML

- At the top we specify the API version to use. If you're using an up-to-date version, Deployment objects are in the apps/v1 API group.
- The .kind field tells Kubernetes we're defining a Deployment.
- The .metadata section is where we give the Deployment a name and labels.
- The .spec section is where most of the action happens.
  - Anything directly below .spec relates to the Pod.
  - Anything nested below .spec.template.spec relates to the containers that will be part of the Pod.
  - At the Pod-level, .spec.replicas tells Kubernetes how may Pod replicas for this Deployment.
  - spec.selector is a list of labels that Pods must have in order for the Deployment to manage them.
  - And .spec.strategy tells Kubernetes how to perform updates to the Deployment.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-pod
        image: nigelpoulton/k8sbook:latest
        ports:
        - containerPort: 8080
```

Deploy and Inspect

- Use kubectl apply to implement it on the cluster.

```
# kubectl apply -f deploy.yml
```

- The Deployment is now instantiated on the cluster.
- We can use the usual `kubectl get` and `kubectl describe` commands to see details of the Deployment.

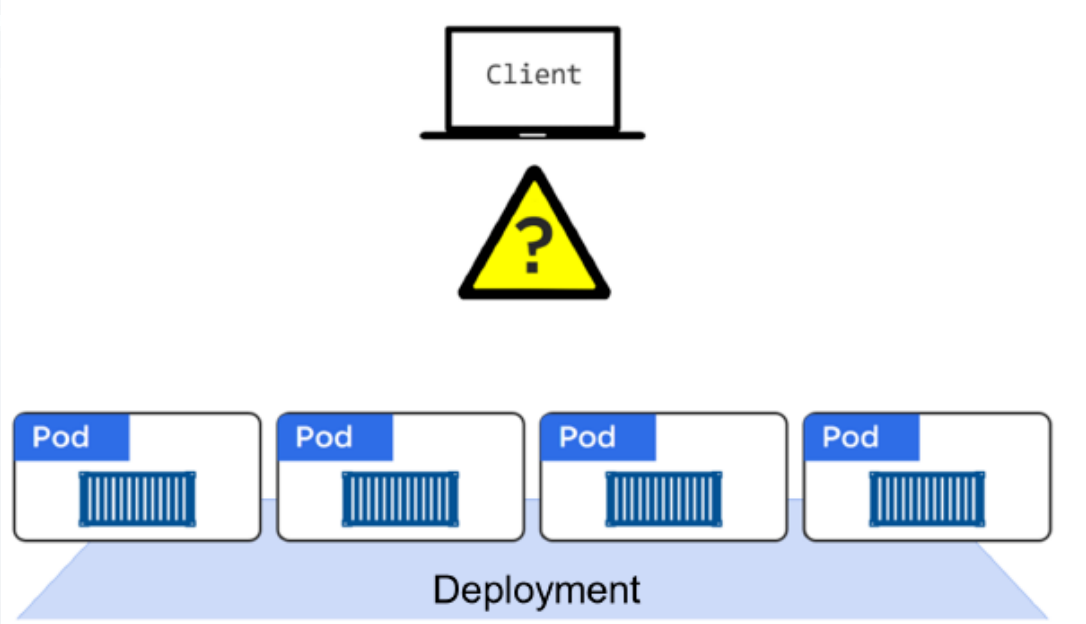
```
# kubectl get deploy hello-deploy
# kubectl describe deploy hello-deploy
```

```
[root@server kubernetes]# kubectl apply -f deploy.yml
deployment.apps/hello-deploy created
[root@server kubernetes]# kubectl get deploy hello-deploy
\NAME            READY   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   10/10   10           10          43s
```

```
[root@server kubernetes]# kubectl get deploy hello-deploy
\NAME           READY   UP-TO-DATE   AVAILABLE   AGE
hello-deploy   10/10   10           10          43s
[root@server kubernetes]# kubectl describe deploy hello-deploy
Name:                   hello-deploy
Namespace:              nsoracle
CreationTimestamp:      Sun, 24 Oct 2021 03:34:17 -0400
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=hello-world
Replicas:               10 desired | 10 updated | 10 total | 10 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        10
RollingUpdateStrategy:  1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=hello-world
  Containers:
   hello-pod:
    Image:        nigelpoulton/k8sbook:latest
    Port:         8080/TCP
```

Service

- Service let you define networking rules for pods based on their labels.
- Whenever a pod with a matching label appears in the Kubernetes cluster, the service will detect it and start using it to handle network requests made to the service.
- A service's ***type*** determines how the service is exposed—whether it is only accessible within the cluster, accessible through a static port on each node, or using a load balancer from a cloud provider.



Creating a Service – Quick Setup

- Let us create the service that will forward network requests to our hostname pods.

```
# vi service.yaml
```

- Create this service with

```
# kubectl apply -f service.yaml
```

- Now, we'll create two pods that the service will use to handle network traffic.
- Pay particular attention to the relationship between the service's selector field and the pods' labels—this is how the service discovers the pods it can use.

```yaml
kind: Service
apiVersion: v1
metadata:
  name: hostname-service
spec:
  type: NodePort
  selector:
    app: echo-hostname
  ports:
    - nodePort: 30163
      port: 8080
      targetPort: 80
```

Creating a Service

- Create a manifest file to deploy two pods

  `# vi svc-deploy.yml`

- Create these two pods with

  `# kubectl apply -f svc-deploy.yml`

```
[root@server ~]# kubectl apply -f svc-deploy.yml
pod/hostname-pod-101 created
pod/hostname-pod-102 created
[root@server ~]#
```

- Get Pods Details

  `# kubectl get all`

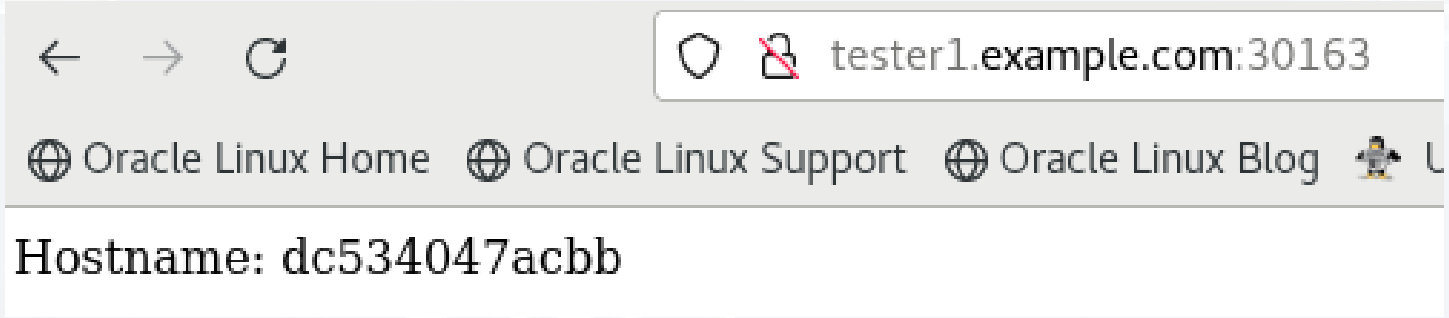
```
[root@server session12]# kubectl get all
NAME                    READY   STATUS    RESTARTS   AGE
pod/hostname-pod-101    1/1     Running   0          74s
pod/hostname-pod-102    1/1     Running   0          74s

NAME                         TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
service/hostname-service     NodePort    10.106.9.65   <none>        8080:30163/TCP   8m9s
service/kubernetes           ClusterIP   10.96.0.1     <none>        443/TCP          3d10h
```

```
kind: Pod
apiVersion: v1
metadata:
  name: hostname-pod-101
  labels:
    # These labels are used by the service to select pods that can handle its network requests
    app: echo-hostname
    app-version: v101
spec:
  containers:
    - name: nginx-hostname
      image: kubegoldenguide/nginx-hostname:1.0.1
      ports:
        # nginx runs on port 80 inside its container
        - containerPort: 80
---
kind: Pod
apiVersion: v1
metadata:
  name: hostname-pod-102
  labels:
    app: echo-hostname
    app-version: v102
spec:
  containers:
    - name: nginx-hostname
      image: kubegoldenguide/nginx-hostname:1.0.2
      ports:
        - containerPort: 80
```

Creating a Service

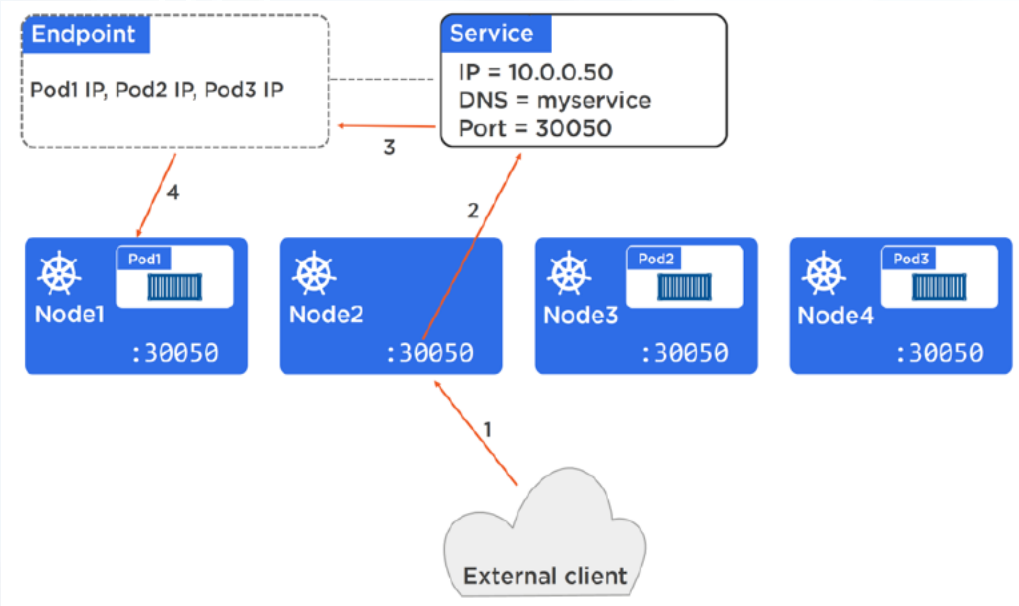
- Access the Pod using any node name/ip address and the port specified in the output



A Comprehensive Lecture is available here.

How it Works?

- The figure shows a NodePort Service where 3 Pods are exposed externally on port 30050 on every node in the cluster.
  1. an external client hits **Node2** on port 30050.
  2. it is redirected to the Service object.
  3. shows that the Service has an associated Endpoint object with an always-up-to-date list of Pods matching the label selector.
  4. shows the client being directed to **pod1** on **Node1**.
- The Service could just as easily have directed the client to pod2 or pod3.
- In fact, future requests may go to other Pods as the Service performs basic load-balancing.



ClusterIP, NodePort and LoadBalancer

- The type property in the Service's spec determines how the service is exposed to the network.
- It changes where a Service is able to be accessed from.
- The possible types are ClusterIP, NodePort, and LoadBalancer
  - ClusterIP
    - The default value.
    - The service is only accessible from within the Kubernetes cluster.
  - NodePort
    - This makes the service accessible on a static port on each Node in the cluster.
  - LoadBalancer
    - The service becomes accessible externally through a cloud provider's load balancer functionality.
    - The Service is associated with the Pods and fronts them with a stable IP, DNS, and port.
    - It also load-balances requests across the Pods.

Kubernetes Service NodePort Example YAML

- This example YAML creates a Service that is available to external network requests.
- We've specified the NodePort value so that the service is allocated to that port on each Node in the cluster.

```yaml
kind: Service
apiVersion: v1

metadata:
  name: hostname-service

spec:
  type: NodePort
  selector:
    app: echo-hostname
  ports:
    - nodePort: 30163
      port: 8080
      targetPort: 80
```

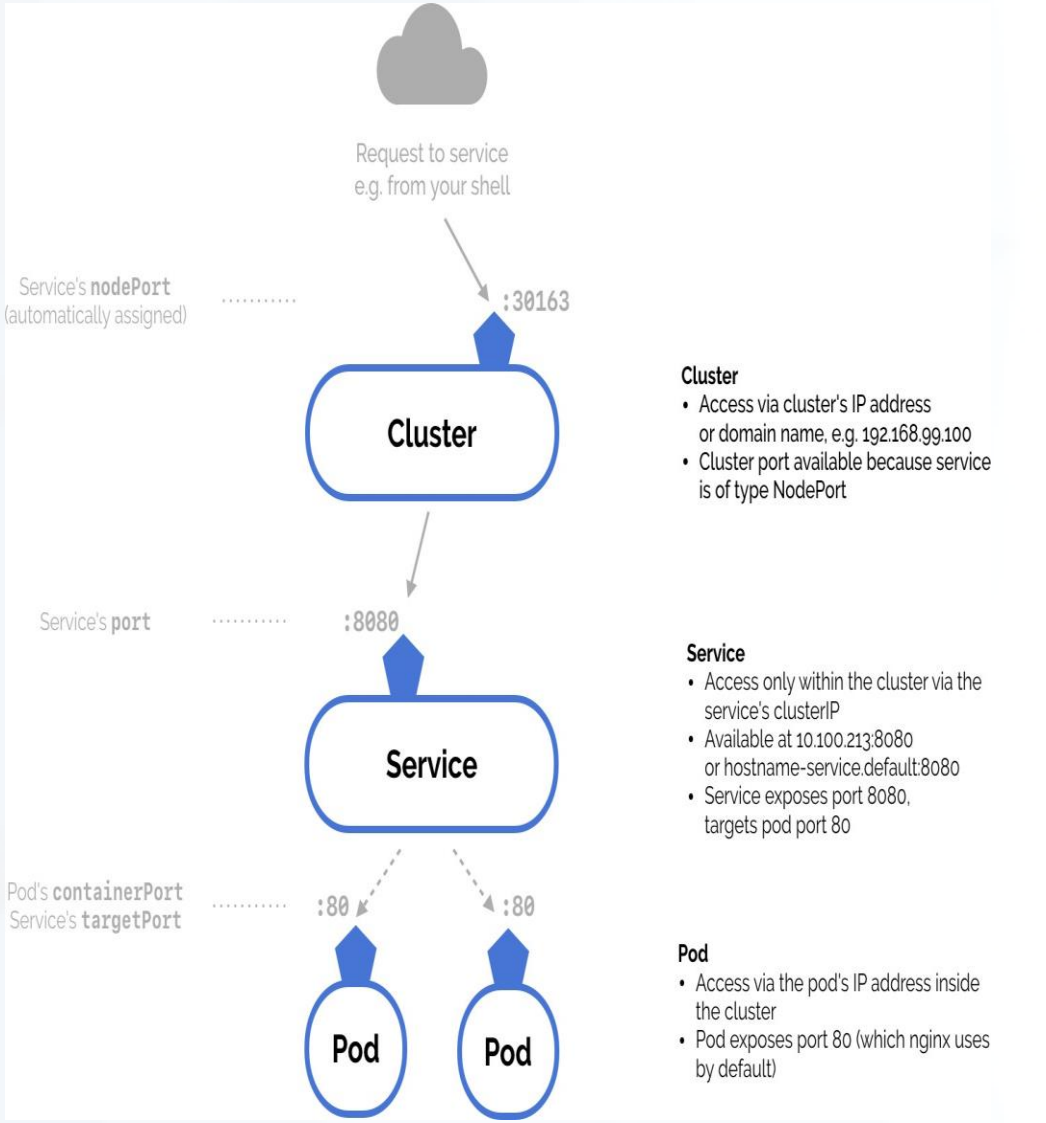
Make the service available to network requests from external clients

Forward requests to pods with label of this value

nodePort
access service via this external port number

port
port number exposed internally in cluster

targetPort
port that containers are listening on



Thank You

Deploy OKE on Oracle Cloud